

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Natan de Jesus Soares



PORTFÓLIO – RELATÓRIO DE AULA PRÁTICA:

Atividade: Atividade 01 – Cadastro de Produtos com Persistência em SQLite

Data: 15 de setembro de 2026

Natan de Jesus Soares

Portforlio – RELATÓRIO DE AULA PRÁTICA
ALGORITMOS E PROGRAMAÇÃO ESTRUTURADA

Trabalho de portfólio apresentado como requisito parcial para a obtenção de pontos para a média
semestral.

Orientador: **VINICIUS MENDES GOMES DA Silva**
Pós Graduado

Data: 15 de setembro de 2026

Sumário

1. INTRODUÇÃO E OBJETIVOS
2. ARQUITETURA E PASSO A PASSO DA IMPLEMENTAÇÃO
 - 2.1 Modelagem da Entidade (Produto.java)
 - 2.2 Gerenciamento do Banco de Dados (ProdutoDbHelper.java)
 - 2.3 Arquitetura de Pacotes
 - 2.4 Lógica de Controle e Validações (MainActivity.kt)
 - 2.5 Teste de Execução e Persistência
3. DESAFIOS TÉCNICOS E SOLUÇÕES
4. RESPOSTAS AOS QUESTIONAMENTOS DO ROTEIRO
5. CONCLUSÃO

1. INTRODUÇÃO E OBJETIVOS

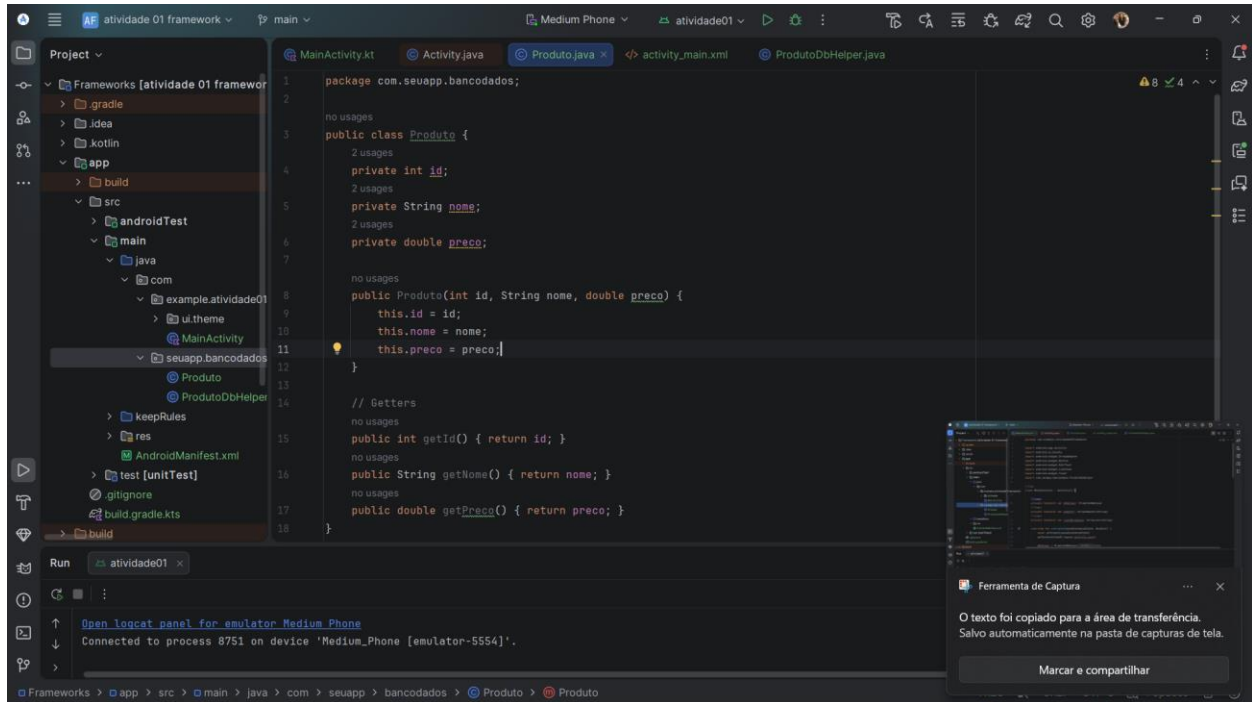
Este relatório apresenta o desenvolvimento de uma aplicação nativa para o sistema operacional Android, focada no cadastro e na listagem de produtos com persistência em banco de dados relacional local.

O objetivo principal do projeto foi integrar a camada de interface gráfica (XML), a lógica de controle e validação de regras de negócio em Kotlin (MainActivity.kt) e a camada de persistência em Java (SQLiteOpenHelper), garantindo o tratamento de exceções de entrada de dados e a retenção das informações.

2. ARQUITETURA E PASSO A PASSO DA IMPLEMENTAÇÃO

2.1 Modelagem da Entidade (Produto.java)

A etapa inicial consistiu na criação do modelo de dados responsável por estruturar as informações da aplicação. A classe Produto foi implementada em Java contendo os atributos primários id, nome e preco, além de seu método construtor e *getters*.

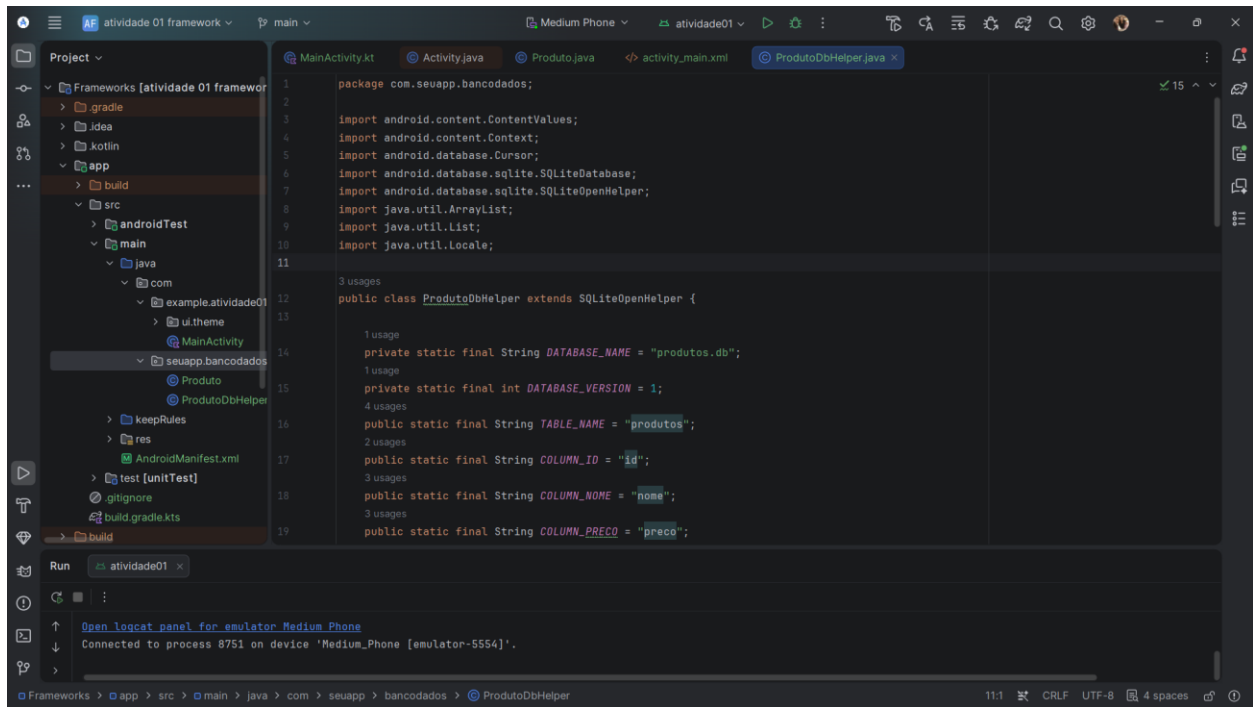


Legenda: Figura 1 – Código fonte da classe de modelo Produto.java.

(Arquivo: Captura de tela 2026-09-15 143035.jpg)

2.2 Gerenciamento do Banco de Dados (ProdutoDbHelper.java)

Para a manipulação da base de dados local, criou-se a classe ProdutoDbHelper herdando de SQLiteOpenHelper. Foram definidas a estrutura do banco (produtos.db), a tabela (produtos) e a implementação dos métodos de CRUD: inserirproduto para gravar os dados e listarProdutos para recuperar os registros e populá-los na interface.



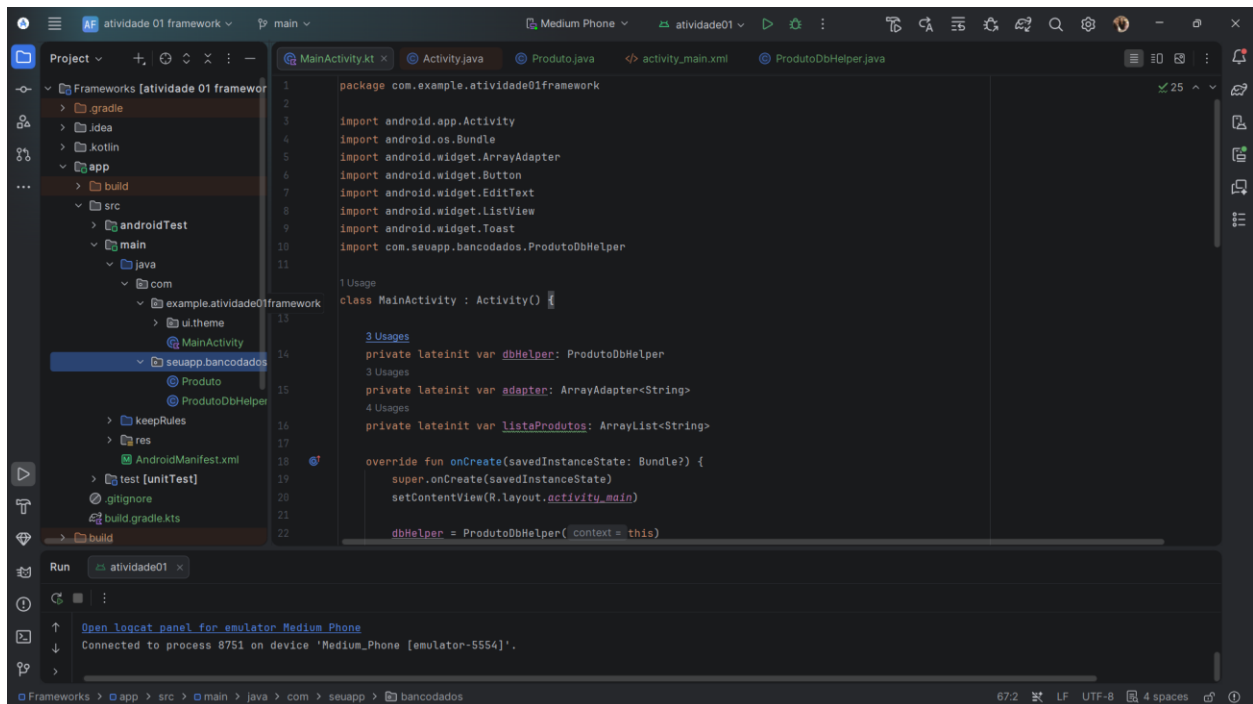
Legenda: Figura 2 – Implementação do helper do banco de dados SQLite.

(Arquivo: Captura de tela 2026-09-15 143133.jpg)

2.3 Arquitetura de Pacotes

A estrutura de diretórios do projeto foi organizada de forma modular para separar as responsabilidades do sistema:

- **com.example.atividade01framework:** Contém as rotinas de interface e controle (MainActivity.kt).
- **com.seuapp.bancodados:** Concentra a camada de persistência e modelo (Produto.java e ProdutoDbHelper.java).

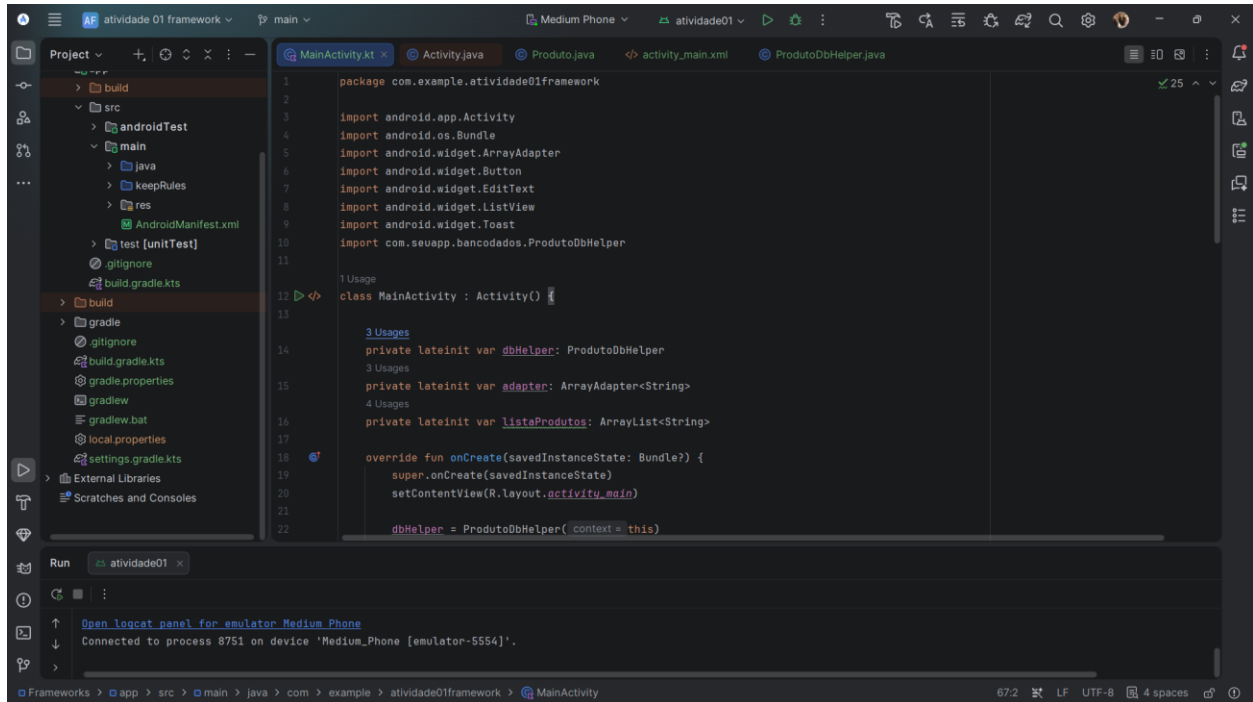


Legenda: Figura 3 – Estrutura de pacotes e arquivos do projeto no Android Studio.

(Arquivo: Captura de tela 2026-09-15 142917.jpg)

2.4 Lógica de Controle e Validações (MainActivity.kt)

A controller principal foi desenvolvida em Kotlin. Ela gerencia o evento de clique do botão de salvamento, aplica as validações necessárias — exigindo nome com no mínimo 3 caracteres e valor numérico positivo — e realiza a chamada aos métodos do repositório SQLite.

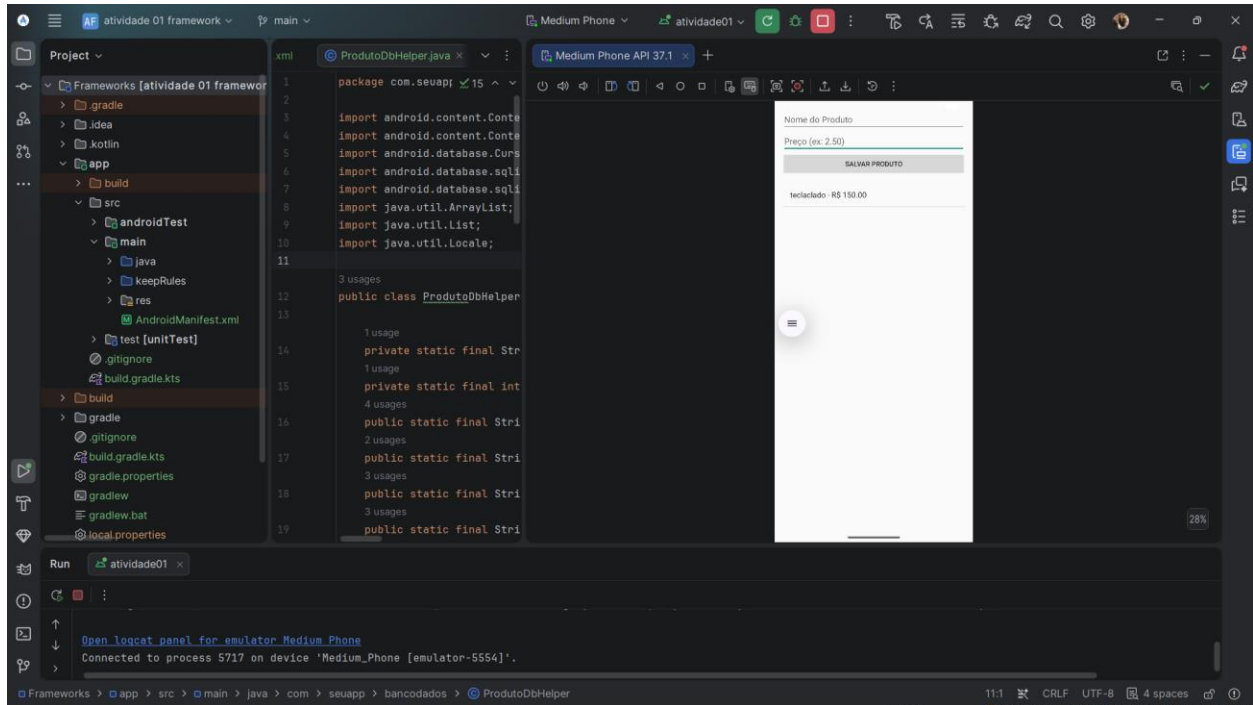


Legenda: Figura 4 – Código fonte do controlador MainActivity.kt.

(Arquivo: Captura de tela 2026-09-15 142818.jpg)

2.5 Teste de Execução e Persistência

O aplicativo foi compilado e executado no emulador Android (*Medium Phone API 37.1*). Durante os testes práticos, realizou-se a inclusão bem-sucedida do item **"teclado"** no valor de **R\$ 150,00**, que passou pelas validações, foi gravado no banco de dados local e exibido dinamicamente na ListView.



Legenda: Figura 5 – Execução da aplicação no emulador com o produto cadastrado.

(Arquivo: Captura de tela 2026-09-15 141757_2.jpg)

3. DESAFIOS TÉCNICOS E SOLUÇÕES

Durante a execução do projeto, surgiram quatro principais imprevistos técnicos que exigiram ajustes pontuais. O primeiro foi um conflito de pacotes (*Package Mismatch*), pois a classe do banco de dados estava no diretório `com.seuapp.bancodados`, diferente do pacote principal, o que acabou impedindo a importação automática da classe `R`; o problema foi resolvido ajustando a declaração do pacote e importando a classe manualmente. Em seguida, enfrentei uma incompatibilidade com o AndroidX ao usar `AppCompatActivity`, que gerava erros de compilação por falta de dependências no `build.gradle.kts`, sendo contornado ao alterar a herança para a classe nativa `Activity`. Na interface gráfica, deparei-me com falhas de renderização causadas por tags não fechadas no `activity_main.xml`, o que exigiu a reestruturação dos elementos `LinearLayout`, `EditText`, `Button` e `ListView`. Por fim, o emulador travou em segundo plano sob o processo PID 17344 e impediu novas compilações, situação solucionada após instalar a imagem de sistema *Intel x86_64* e realizar a inicialização limpa (*Cold Boot Now*) pelo Device Manager.

4. RESPOSTAS AOS QUESTIONAMENTOS DO ROTEIRO

1. Com base nos seus conhecimentos, qual a vantagem de usar o SQLite em aplicações mobile em comparação ao armazenamento somente em memória? Justifique.

Resposta: A principal vantagem do SQLite é a persistência dos dados. O armazenamento somente em memória (usando variáveis, listas ou arrays convencionais) é volátil, ou seja, todos os dados são perdidos assim que o aplicativo é fechado ou o dispositivo é reiniciado. O SQLite permite criar um banco de dados relacional embutido diretamente no armazenamento interno do celular, garantindo que os registros cadastrados continuem disponíveis nas próximas vezes em que o usuário abrir o aplicativo. Além disso, o SQLite lida melhor com grandes volumes de dados através de consultas estruturadas (SQL).

2. Considerando a persistência local e o uso de SQLiteOpenHelper, o que seria mais indicado para aplicações que requerem sincronização com dados online? Reflita sobre sua resposta.

Resposta: Embora o SQLiteOpenHelper seja excelente para acesso offline rápido, aplicativos que requerem sincronização com a nuvem costumam adotar bibliotecas mais modernas e robustas. No ecossistema nativo atual, o Room Database (uma abstração moderna do próprio SQLite no Android) em conjunto com APIs REST (consumidas via Retrofit) é a abordagem padrão. O Room facilita a integração com arquiteturas reativas (como LiveData ou Flow). Outra solução muito indicada para sincronização em tempo real (online/offline) é o uso de bancos de dados NoSQL gerenciados (BaaS), como o Firebase Realtime Database ou Cloud Firestore, que já possuem cache local e sincronização automática nativa de dados de forma transparente, exigindo muito menos código manual de sincronização do que o SQLite puro.

5. CONCLUSÃO

A realização desta atividade permitiu aplicar de forma prática os conceitos de manipulação de interfaces móveis, regras de validação em Kotlin e armazenamento persistente com SQLite. Apesar das divergências de pacotes e travamentos de ambiente durante o processo, todas as falhas foram devidamente diagnosticadas e resolvidas, culminando em uma aplicação funcional, estável e que atende a todos os requisitos propostos.

